Can a Small Convolutional Network Still Read Blurred and Pixelated Digits? A Replication of McPherson et al. (2016)

Shashwat Deep
Computer Science and Engineering (Data Science)
Manipal University Jaipur
Jaipur, Rajasthan, India
https://orcid.org/0009-0008-3580-2026

***Abstract*—People blur or pixelate faces, license plates, and text, assuming that what cannot be recognized is protected. McPherson, Shokri, and Shmatikov showed that neural networks trained on obfuscated images still recognize their content. In a small replication on handwritten digits, I examined whether a much cheaper version of their attack still works. I trained a small convolutional network for two epochs on pixelated, blurred, or fully masked MNIST digits and compared it with the same network trained on clean digits across ten seeds. The clean-trained model fell from 98.6% to 13.4% (near chance) at the coarsest pixelation and to 36.4% at the strongest blur. Trained on obfuscated digits, the network reached 51.2% and 89.3%. Ten epochs raised the blur result to 94.9% but the coarsest pixelation result only to 51.9%, staying within a point of McPherson et al. and Mahloujifar et al. at both training lengths. Strong blurring and pixelation hurt the clean-trained model far more than the obfuscation-trained model, although the coarsest pixelation still cost the trained model 47 points. The mask, which removes the digit, kept accuracy near chance (11.4%). Rerunning the deterministic experiment reproduced every result exactly.**

Keywords—image obfuscation, pixelation, Gaussian blur, privacy, replication, convolutional neural networks, MNIST

I. Introduction

Blurring and pixelation are standard methods for hiding parts of an image. News photographs pixelate or blur the faces of police officers and crime victims, Google Street View obscures faces and license plates, YouTube provides a tool that blurs every face in a video, and similar techniques redact text in documents [1]. These methods are popular because they leave the rest of the image intact and look less jarring than a black box. The protection they offer rests on an informal assumption: content is hidden because human viewers can no longer recognize it [1].

McPherson, Shokri, and Shmatikov [1] demonstrated that this assumption fails against machine learning. They trained standard neural networks directly on obfuscated images, and the networks recognized faces, objects, and handwritten digits far above chance. On MNIST digits pixelated with 16×16-pixel windows, for example, their network was correct 52% of the time, whereas random guessing gives 10%.

This study investigates whether a much cheaper version of this attack still works: can a small convolutional neural network (CNN), trained for only two epochs, read blurred and pixelated digits, and how does that compare with an obfuscation that removes the digit entirely? Mahloujifar et al. [2] recently compared clean-trained and obfuscation-trained models on MNIST, including Gaussian blur, making that comparison familiar. Relative to both papers, this replication introduces four additions: independent results after two and ten epochs that agree with both papers at the coarsest pixelation and show where longer training helps; a solid-mask control that neither paper ran on MNIST; both attackers tested at milder settings than those used by [2] (namely 2- and 4-pixel blocks and blur with $\sigma = 1$ and 2); and the spread across ten seeds, which neither paper reports. The experiments use MNIST [3] and a CNN with two convolution layers, trained with Adam [4] and dropout [5] in PyTorch [6].

II. Background

A. Defeating Obfuscation With Deep Learning

McPherson et al. [1] evaluated three obfuscation techniques. Mosaicing (pixelation) divides an image into a grid of square windows and sets each window to its average color. For blurring, they used YouTube's automatic face blurring. P3, a privacy-preserving photo sharing system, splits a JPEG image into a public part and an encrypted secret part; the secret part holds the DC coefficient of each 8×8 block and every AC coefficient larger than a threshold. They tested mosaicing and P3 on four datasets (MNIST digits, CIFAR-10 objects, AT&T faces of 40 people, and FaceScrub faces of 530 celebrities) and YouTube blurring on AT&T faces.

Their attack trains the recognition model on images that are already obfuscated. The attacker knows the obfuscation method, applies it to labeled training images, and trains an ordinary classifier on the result. The network learns which features survive obfuscation, eliminating the need to reconstruct the hidden image or reverse-engineer leaks. For MNIST, they padded images to 32×32 and used a CNN with two 5×5 convolution layers (32 and 64 filters), a 200-unit fully connected layer, and 0.5 dropout, trained via stochastic gradient descent for 100 to 200 epochs. It reached 98.71% on clean digits and 98.49%, 96.17%, 83.42%, and 52.13% with 2×2, 4×4, 8×8, and 16×16 mosaic windows. On faces blurred by YouTube, their AT&T model achieved 57.75% accuracy (compared to 2.5% random guessing). Each model was trained and tested on a single version of the data, so [1] did not test clean-trained models on obfuscated images.

B. Related Work

The underlying concept predates deep learning. Newton et al. [7] attacked de-identified faces using an eigenface recognizer. Pixelation barely hindered an unadapted recognizer, which matched 99% of faces (against 0.5% guessing). Eye bars and T-shaped masks limited matching to 1%–2%. When the attacker applied the same masking to training and gallery faces (parrot recognition), accuracy reached 100%; blacking out the entire face kept performance at random guessing levels. However, both 99% and 100% scores matched each probe against a gallery containing the exact same photograph. Zhou et al. [8] compared clean-trained

© 2026 Shashwat Deep. Licensed under CC BY 4.0 (https://creativecommons.org/licenses/by/4.0/). DOI: 10.5281/zenodo.23692156.
Version 1.0.0, October 2026.

models with retrained and fine-tuned models under motion blur, defocus blur, and noise, evaluating camera fault robustness rather than privacy. Both approaches sharply reduced error on MNIST and CIFAR-10, though fine-tuning on ImageNet yielded modest gains (top-1 error decreasing from 53.1% to 47.7%). Fan [9] applied the attack from [1] to MNIST to test differentially private pixelation, which restricted the trained attacker to 16%–23% accuracy at 16-pixel cells, compared to 52.13% for plain mosaicing in [1]; Fan evaluated only the trained attacker. For faces, Tekli et al. [10] re-identified 692 of 854 blurred celebrities (81%) using a recognizer trained on blurred photos, with the attack remaining effective, at lower rates, even without knowledge of the exact blur kernel. Among 53 human viewers, Li et al. [11] found that a solid block over a person hid identity more effectively than blurring, though viewers preferred it least. Oh et al. [12] recognized individuals in social media photos even with blacked-out heads, using body and environmental context. For text, Hill et al. [13] recovered mosaiced and blurred words using hidden Markov models.

The closest prior work is Privacy Blur by Mahloujifar et al. [2]. On MNIST, they report both attackers: a clean-trained classifier used to score reconstructed images, and a classifier trained on obfuscated digits (termed a discrimination attack). Their settings include pixelation to 4×4, 2×2, and 1×1 blocks and Gaussian blur at two radii, alongside a reversal attack that inverts low-precision blurs from image libraries. They also analyze cropping (equivalent to a solid mask) and prove it leaves no distinguishable features, though they do not test it on MNIST. They conclude that blur offers the weakest privacy while pixelation at appropriate granularities protects realistic data like house numbers, though MNIST remains vulnerable to discrimination attacks.

C. The MNIST Dataset

MNIST [3] contains 70,000 grayscale images of handwritten digits. LeCun et al. compiled it from two NIST collections (one written by Census Bureau employees and one by high school students), scaled each digit to fit a 20×20 box, and centered it by center of mass within a 28×28 image [3].

D. Convolution and Pooling

A convolution layer slides a small grid of learned weights (a filter or kernel) across an image. At each position, it computes the weighted sum of underlying pixels plus a bias, passing the result through a nonlinearity such as the rectified linear unit (ReLU), max(0, x). Because weights are shared across positions, a filter detects features like edges or stroke endings regardless of location, requiring far fewer parameters than fully connected layers. A layer with k filters produces k feature maps. A pooling layer then summarizes feature map windows (typically 2×2) via maximum (max pooling) or mean (average pooling), halving height and width. Pooling reduces computation and local translation sensitivity. LeCun et al. [3] describe these principles as local receptive fields, shared weights, and subsampling. Pixelation itself acts as a pooling operation: averaging non-overlapping b×b blocks is average pooling with a b×b window and stride b, followed by nearest-neighbor upsampling back to the original dimensions.

III. Method

A. Data

I used the MNIST dataset [3] via torchvision: 60,000 training images and 10,000 test images of handwritten digits, each 28×28 pixels. Pixel values were scaled to [0, 1] with white strokes on a black background. No other preprocessing, data augmentation, or validation split was applied.

B. Obfuscations

Each obfuscation was applied identically to training and test images (Fig. 1). Pixelation with block size $b \in \{2, 4, 7, 14\}$ replaces every non-overlapping b×b block with its mean value, then upsamples back to 28×28 by repeating values. Because 28 is divisible by each b, images retain 14×14, 7×7, 4×4, or 2×2 blocks (representing 196, 49, 16, or 4 numbers). Gaussian blur with standard deviation $\sigma \in \{1, 2, 4\}$ pixels is applied via 1D convolution along rows and columns. Kernels extend 3σ pixels on each side (7, 13, and 25 taps), sum to one, and use reflection padding at borders. The solid mask serves as a control condition, setting every pixel to zero (background value) so no trace of the digit remains.

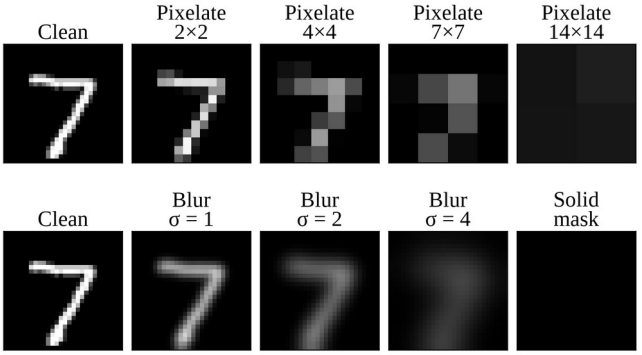


Fig. 1. The first MNIST test image, a 7, under each condition. Top row: pixelation with blocks of 2×2 to 14×14 pixels. Bottom row: Gaussian blur with σ = 1, 2 and 4 pixels, and the solid mask. Both rows start from the same clean image.

C. Model

The model is a small CNN implemented in PyTorch [6]. Two 3×3 convolution layers with 32 and 64 filters (stride 1, padding 1, ReLU activation) are each followed by 2×2 max pooling, reducing the 28×28 input to 14×14 and then 7×7. The resulting 64×7×7 = 3,136 values feed a fully connected layer of 128 ReLU units with 0.25 dropout [5] (zeroing units with probability 0.25 during training, retention probability p = 0.75). The final layer has 10 outputs corresponding to digit classes. The model contains 421,642 parameters.

D. Training

Models were trained using Adam [4] with a learning rate of 0.001, default PyTorch hyperparameters (β_1 = 0.9, β_2 = 0.999, $\varepsilon = 10^{-8}$) [4], batch size 128, cross-entropy loss, and 10 epochs (469 parameter updates per epoch). Learning rates remained constant without weight decay or early stopping. Both attackers were evaluated after epochs 2 and 10. Evaluation draws no random numbers, so models at epoch 2 (938 updates) represent an exact two-epoch run.

E. Two Attackers

The first attacker, trained on clean images, is unadapted to obfuscation: one model per seed is trained on clean digits and evaluated across all conditions. The second attacker, trained on obfuscated images, follows [1] by training separate models for each obfuscation condition using labeled obfuscated training data. Under the clean condition, both attackers are identical.

F. Measure and Setup

Accuracy on the 10,000 test images serves as the primary metric. Random guessing yields 10%, while always predicting

"1" (the most frequent test digit) yields 11.35% (taken as the baseline chance level). Following the public notebook and results [14], runs were repeated across ten seeds (0 to 9) using an NVIDIA T4 GPU in Google Colab with PyTorch 2.11 and deterministic GPU algorithms. Results represent means over ten seeds with sample standard deviations (divided by $n - 1 = 9$), except for two CPU validation checks in Section V (three and five seeds). Within each seed, all models share initial weights and batch ordering, ensuring differences stem solely from obfuscation conditions or training epochs. Differences between conditions, models or epochs are therefore taken within a seed before averaging. Ninety-five percent confidence intervals (CIs) derive from the *t*-distribution with $n - 1$ degrees of freedom (mean ± 0.72 × SD in Table I). A complete rerun of all 90 models reproduced per-seed results bit for bit.

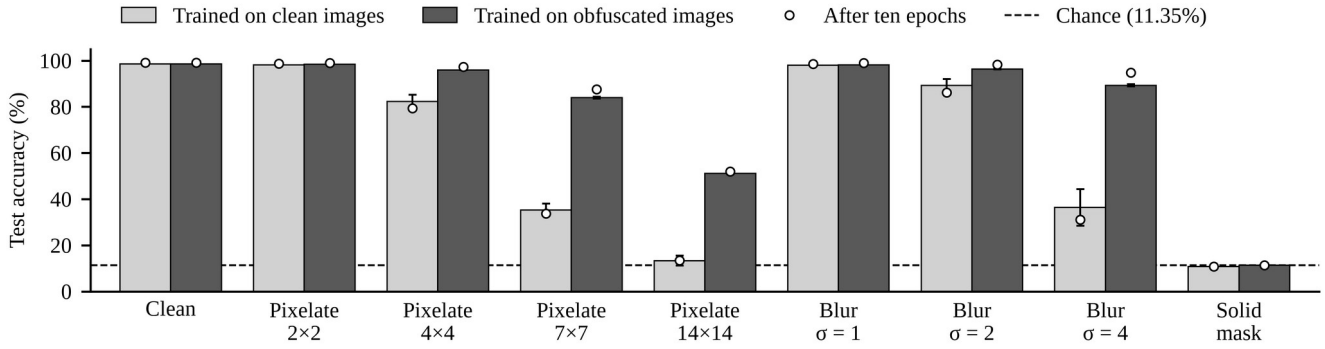


Fig. 2. Test accuracy under each condition for the model trained on clean images (light bars) and the model trained on obfuscated images (dark bars), after two epochs. Bars show the mean over ten seeds and error bars the 95% CI; circles mark the means after ten epochs. The dashed line marks chance (11.35%).

IV. Results

Table I lists test accuracies.

TABLE I. Test Accuracy (%), Mean ± SD Over Ten Seeds

Condition	Trained on clean images, 2 epochs	Trained on obfuscated images, 2 epochs	Trained on obfuscated images, 10 epochs
Clean	98.62 ± 0.09	98.62 ± 0.09	99.19 ± 0.05
Pixelate 2×2	98.12 ± 0.26	98.39 ± 0.14	99.04 ± 0.06
Pixelate 4×4	82.24 ± 4.21	95.96 ± 0.12	97.35 ± 0.10
Pixelate 7×7	35.34 ± 3.79	83.98 ± 0.64	87.59 ± 0.36
Pixelate 14×14	13.36 ± 3.07	51.15 ± 0.40	51.92 ± 0.26
Blur σ = 1	97.97 ± 0.23	98.19 ± 0.11	99.01 ± 0.07
Blur σ = 2	89.29 ± 3.89	96.35 ± 0.23	98.28 ± 0.18
Blur σ = 4	36.40 ± 11.00	89.28 ± 0.69	94.86 ± 0.18
Solid mask	10.79 ± 0.93	11.35 ± 0.00	11.35 ± 0.00

A. Clean-Trained Model

On clean images, the clean-trained model reached 98.62%. Mild settings caused minimal impact (98.12% at 2×2 pixelation, 97.97% at σ = 1), but performance dropped rapidly at stronger settings. Pixelation reduced accuracy to 82.24% at 4×4, 35.34% at 7×7, and 13.36% at 14×14 (CI 11.16 to 15.56, not clearly above chance). Blurring reduced accuracy to 89.29% at σ = 2 and 36.40% at σ = 4. Seed variance was high at middle levels (SD of 11.00 points at σ = 4), showing that an unadapted model's performance on unseen distortions fluctuates significantly between runs.

B. Obfuscation-Trained Model

At mild settings, both models performed within half a point of each other. At stronger settings, the obfuscation-trained model significantly outperformed the clean-trained model, reaching 95.96% at 4×4 pixelation, 83.98% at 7×7, and 51.15% at 14×14 (4.5 times chance accuracy from images containing only four numbers). Under blur, it reached 96.35% at σ = 2 and 89.28% at σ = 4 (only 9.33 points below clean performance). Improvements over the clean-trained model were statistically significant across all strong settings, ranging from 7.06 points at σ = 2 to 52.89 points at σ = 4. The obfuscation-trained model's standard deviation was at most 0.69 points in every condition.

C. Solid Mask

The solid mask held both models near chance: 10.79 ± 0.93% for the clean-trained model and 11.35 ± 0.00% for the mask-trained model. Because every masked image is identical, models must output a constant prediction, yielding an accuracy equal to the proportion of test images matching that label. The mask-trained model learned to predict "1" (the most frequent training digit, 6,742 of 60,000 images), matching 1,135 test images (exactly 11.35%) across all seeds and epochs. After two epochs, clean-trained models predicted "1" in seven seeds and "0", "5", or "8" in single seeds.

D. Ten Epochs

After 10 epochs, the obfuscation-trained model improved across all conditions except the mask (Table I). The largest gains occurred at σ = 4 (+5.58 points to 94.86%) and 7×7 pixelation (+3.62 points to 87.59%). At 14×14 pixelation, accuracy increased by 0.76 points to 51.92%. The clean-trained model reached 99.19 ± 0.05% on clean images, with minor gains at mild settings (+0.65 points at 2×2, +0.53 at σ = 1), but showed no significant improvements under stronger obfuscations (changes ranging from −5.28 to +0.13 points).

V. Discussion

Visual inspection suggests that blurring and pixelation make digits difficult for humans to read: at 14×14 pixelation, a 7 resembles four nearly identical gray squares, and at σ = 4, it appears as a faint smudge. These distortions severely impair models trained solely on clean digits, dropping accuracy to 13% at 14×14 and 36% at σ = 4. Adapted attackers perform substantially better. Trained on obfuscated examples, the same CNN correctly classified 89% of σ = 4 digits and half of the 14×14 digits after two epochs on a single GPU. Although obfuscation alters visual appearance, sufficient discriminative structure survives for a classifier to exploit, regardless of human readability. Retention varies: the trained model lost 9.33 points at σ = 4 but 47.46 points at 14×14. The solid mask removes the digit entirely, preventing training utility and forcing models to guess the majority class.

My pixelation results align closely with McPherson et al. [1] and Mahloujifar et al. [2]. Because [1] used 32×32 padded images and my experiments used 28×28 images, levels are

compared by blocks per side (matching [2]): 16 vs 14, 8 vs 7, and 4 and 2 in both. At these four levels, the model in [1] achieved 98.49%, 96.17%, 83.42%, and 52.13%, compared to my 98.39%, 95.96%, 83.98%, and 51.15% (clean scores: 98.71% vs 98.62%). At 4×4 and 2×2 grids, [2] reported 83.54% and 51.16%. Grid alignment differs slightly because [1] added a 2-pixel border, dividing digits into unequal strips (6, 8, 8, 6 pixels) at 4 blocks per side. On that grid, a CPU check of my model (five seeds) yielded 79.71% after two epochs (3.7 points below [1]) and 83.58% after ten epochs (matching [1]). On my unpadded grid (shared with [2]), ten epochs raised accuracy to 87.59% (about 4 points above [2]), indicating that 2-epoch results do not represent the absolute limit of what trained attackers extract from 16 block averages. At 2 blocks per side, all three setups agree within one point despite differences in architectures and training durations ([1] used 100–200 epochs). Ten epochs added only 0.76 points to 51.92%, remaining within a point of both studies. Even on [2]'s coarsest single-block grid, a trained model achieved 22.06% accuracy (twice chance), which [2] attributes to MNIST's high dataset clarity. Clean-trained models collapsed in both studies: [2] reports 23.5% and 14.7% at 4×4 and 2×2 grids, compared to my 35.34% and 13.36%. The 23.5% figure falls below all 20 of my seed/epoch values at 7×7 (lowest 26.77%) and outside the 95% prediction interval (26.35% to 44.32%), suggesting differences in model architecture or preprocessing rather than seed variance.

For blurring, [2] sets small radii to 1/10 of "the maximum diagonal" and large radii to 1/7, with PIL treating radius as Gaussian standard deviation [15]. Reading this as the diagonal of the 28×28 image, which is my assumption rather than a definition in [2], yields $\sigma \approx 4$ for the small radius. At this setting, their trained model achieved 97.58% (vs my 89.28%) and their clean-trained model 47.8% (vs my 36.40%). Ten epochs raised my $\sigma = 4$ result to 94.86% (closing two-thirds of the gap), while clean-trained performance did not improve (31.11%), showing that longer training primarily benefits obfuscation-trained models at strong settings. The remaining 2.7-point gap is not due to blur implementation differences. Although PIL approximates Gaussians via box filters, edge padding, and 8-bit rounding, a CPU check (three seeds, ten epochs) showed PIL blur yielded only 0.49 points less than my implementation (CI −0.92 to −0.05). The discrepancy likely stems from architectural or training differences; [2] describes its model only as a CNN with 98.8% clean accuracy. Code from Yang et al. [16] (cited by [2]) uses the longer side of the bounding box as the diagonal, yielding radii of 2.8 and 4.0 for 28×28 images. Under that interpretation, $\sigma = 4$ corresponds to their large radius, where their trained model achieved 95.75% (within one point of my 10-epoch 94.86%) and their clean-trained model 12.94% (below my 36.40%).

Blurring preserved privacy less effectively than pixelation in this experiment. Blur at $\sigma = 4$ and 7×7 pixelation caused comparable damage to clean-trained models on average (36.40% and 35.34%; paired difference 1.06 points, CI −5.21 to 7.32), though individual seeds varied by up to 14 points. However, after retraining, blur permitted 5.31 points higher accuracy than pixelation (CI 4.67 to 5.94), increasing to 7.27 points after ten epochs (CI 7.03 to 7.51). While 7×7 pixelation retains only 16 numbers per image, blur preserves all 784 pixels through smoothing, probably leaving more shape information for trained models. For practical redaction, these findings reinforce [1]: content requiring privacy must be removed entirely (e.g., via solid masking, which also hid identity best from human viewers [11]). Image obfuscation should always be tested against models trained on obfuscated data rather than evaluated solely by visual inspection.

VI. Limitations

This study evaluates handwritten digits. MNIST features ten centered classes on plain backgrounds, whereas faces, license plates, and documents exhibit substantially higher variability. While these exact figures do not generalize directly, [1] observed the same pattern on face data. The model is small and trained for at most ten epochs. Since 10 epochs outperformed 2 epochs across all obfuscations except the mask, obfuscation-trained accuracies represent lower bounds on attacker capabilities. At 14×14 pixelation, however, additional epochs produced minimal change.

On my reading of the blur radius of [2], 2.7 points of the gap remain unexplained, though blur implementation was ruled out. Confidence intervals reflect only seed variation on a single GPU architecture and software stack. Seeds 0 to 2 agreed within 0.71 points (0.20 for obfuscation-trained models) with an earlier non-deterministic three-seed run. P3 encryption was not tested. Human readability of obfuscated digits was not measured; claims regarding human recognition rely on visual inspection. Finally, following [1], attackers possessed exact knowledge of test block sizes and σ values, meaning parameter mismatches between training and testing were not evaluated.

VII. Conclusion

A small CNN trained for two epochs on obfuscated MNIST digits successfully classified 51% of digits pixelated to four blocks, matching McPherson et al. [1] and Mahloujifar et al. [2] within one point, and remaining within one point after ten epochs. On digits blurred with $\sigma = 4$, accuracy reached 89% (95% after ten epochs). Only the solid mask, which removes the digit entirely, maintained accuracy at baseline majority-class guessing rates. Except at the coarsest pixelation levels, blurring and pixelation altered visual appearance far more than they degraded machine readability. Content requiring confidentiality must therefore be removed.

Acknowledgment

Claude (Anthropic), an AI assistant, wrote the experiment code, suggested the outline of this paper and drafted an earlier version of its text, which I rewrote. A review of an earlier draft by Claude found errors that this version corrects; I checked each against its source. I planned the obfuscation levels, ran the notebooks on Google Colab and the two CPU checks in Section V on my own laptop, verified the results, and take full responsibility for the final text.

References

[1] R. McPherson, R. Shokri, and V. Shmatikov, "Defeating image obfuscation with deep learning," arXiv:1609.00408, 2016.

[2] S. Mahloujifar, N. Kokhlikyan, C. Guo, and K. Chaudhuri, "Privacy Blur: Quantifying privacy and utility for image data release," arXiv:2512.16086, Dec. 2025.

[3] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.

[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[5] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014.

[6] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2019.

[7] E. M. Newton, L. Sweeney, and B. Malin, “Preserving privacy by de-identifying face images,” *IEEE Trans. Knowl. Data Eng.*, vol. 17, no. 2, pp. 232–243, Feb. 2005, doi: 10.1109/TKDE.2005.32.

[8] Y. Zhou, S. Song, and N.-M. Cheung, “On classification of distorted images with deep convolutional neural networks,” in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2017, pp. 1213–1217, doi: 10.1109/ICASSP.2017.7952349.

[9] L. Fan, “Image pixelization with differential privacy,” in *Data and Applications Security and Privacy XXXII* (Lecture Notes in Computer Science, vol. 10980). Cham, Switzerland: Springer, 2018, pp. 148–162, doi: 10.1007/978-3-319-95729-6_10.

[10] J. Tekli, B. Al Bouna, G. Tekli, and R. Couturier, “A framework for evaluating image obfuscation under deep learning-assisted privacy attacks,” *Multimedia Tools Appl.*, vol. 82, pp. 42173–42205, 2023, doi: 10.1007/s11042-023-14664-y.

[11] Y. Li, N. Vishwamitra, B. P. Knijnenburg, H. Hu, and K. Caine, “Blur vs. block: Investigating the effectiveness of privacy-enhancing obfuscation for images,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops (CVPRW)*, 2017, pp. 1343–1351, doi: 10.1109/CVPRW.2017.176.

[12] S. J. Oh, R. Benenson, M. Fritz, and B. Schiele, “Faceless person recognition: Privacy implications in social media,” in *Computer Vision – ECCV 2016* (Lecture Notes in Computer Science, vol. 9907). Cham, Switzerland: Springer, 2016, pp. 19–35, doi: 10.1007/978-3-319-46487-9_2.

[13] S. Hill, Z. Zhou, L. Saul, and H. Shacham, “On the (in)effectiveness of mosaicing and blurring as tools for document redaction,” *Proc. Priv. Enhanc. Technol.*, vol. 2016, no. 4, pp. 403–417, 2016, doi: 10.1515/popets-2016-0047.

[14] S. Deep, “MNIST obfuscation replication,” version 1.0.0, GitHub, Oct. 2026. [Online]. Available: https://github.com/shashwat-deep/mnist-obfuscation-replication. Archived with this paper on Zenodo, doi: 10.5281/zenodo.23692156.

[15] Pillow developers, “ImageFilter module,” Pillow documentation. Accessed: Oct. 16, 2026. [Online]. Available: https://pillow.readthedocs.io/en/stable/reference/ImageFilter.html

[16] K. Yang, J. H. Yau, L. Fei-Fei, J. Deng, and O. Russakovsky, “A study of face obfuscation in ImageNet,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 25313–25330. [Online]. Available: https://github.com/princetonvisualai/imagenet-face-obfuscation